Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Animasyon ve Zamanlayıcılar

PyGame ile Oyun Geliştirme – Hafta 8

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

1. Frame Animasyonu (sprite sheet, kesme, zamanlama)
2. AnimatedSprite Sınıfı Tasarımı
3. State Machine ile Animasyon Durumları
4. Zamanlayıcılar (get_ticks, USEREVENT, set_timer)
5. Cooldown Sistemi
6. Görsel Efektler (parçacık, patlama, screen shake)
7. Özet ve Uzay Kaçışı v4

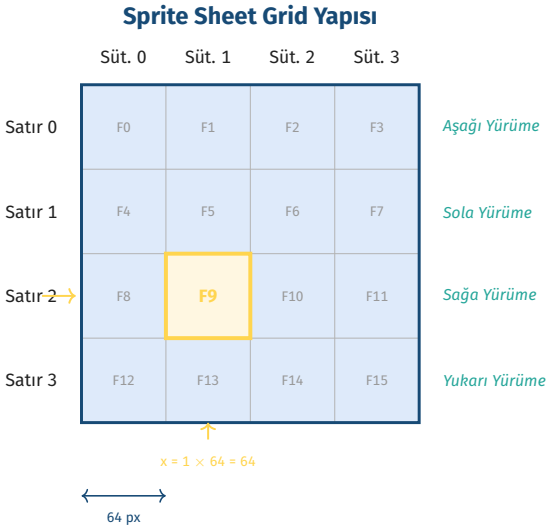
Frame Animasyonu

Sprite Sheet Nedir?

Tanım

Tüm animasyon karelerinin tek bir görselde düzenli olarak sıralanmış halidir.

- Her hücre = 1 frame
- Satırlar = farklı animasyonlar
- Sütunlar = animasyon kareleri



subsurface() ile Frame Kesme

```
# Sprite sheet yukle
sheet = pygame.image.load("karakter.png").convert_alpha()

# Frame boyutlari
FRAME_GEN = 64
FRAME_YUK = 64

# Belirli bir frame'i kes (satir=0, sutun=2)
frame_rect = pygame.Rect(2 * FRAME_GEN, 0, FRAME_GEN, FRAME_YUK)
frame = sheet.subsurface(frame_rect)
```

Önemli

`convert_alpha()` sprite sheet yüklendikten **hemen sonra** çağrılmalı.

Zamana Bağlı Animasyon

```python
class BasitAnimasyon:
    def __init__(self, frameler, frame_suresi=100):
        self.frameler = frameler
        self.frame_suresi = frame_suresi
        self.mevcut_frame = 0
        self.son_guncelleme = pygame.time.get_ticks()

    def guncelle(self):
        simdi = pygame.time.get_ticks()
        if simdi - self.son_guncelleme >= self.frame_suresi:
            self.mevcut_frame = (self.mevcut_frame + 1) % len(self.frameler)
            self.son_guncelleme = simdi
```

Loop vs One-Shot

Loop: Başa dön → yürüme, idle

One-Shot: Son frame'de dur → patlama, ölüm

AnimatedSprite Sınıfı

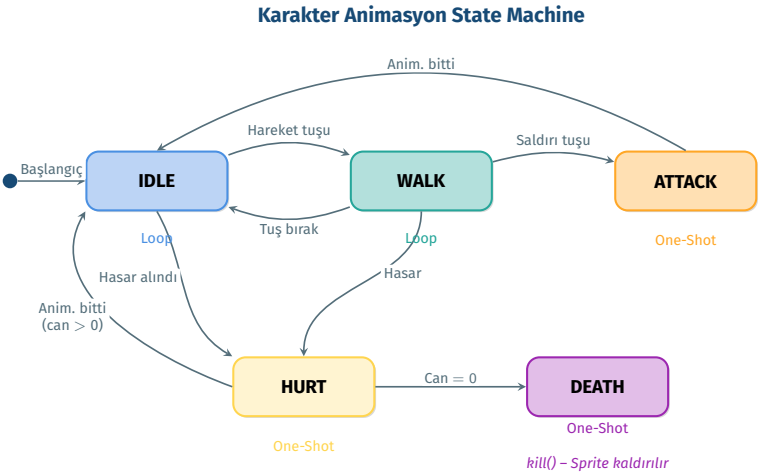
AnimatedSprite – Genel Amaçlı Sınıf

```
class AnimatedSprite(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.animasyonlar = {}
        self.mevcut_durum = None
        self.frame_indeks = 0

    def animasyon_ekle(self, ad, frameler, sure=100):
        self.animasyonlar[ad] = {"frameler": frameler, "sure": sure}

    def durum_degistir(self, yeni_durum):
        if yeni_durum != self.mevcut_durum:
            self.mevcut_durum, self.frame_indeks = yeni_durum, 0
```

State Machine



Animasyon Durumları

Durum	Mod	Geçiş
idle	Loop	Hareket $\rightarrow$ walk
walk	Loop	Dur $\rightarrow$ idle, Saldır $\rightarrow$ attack
attack	One-Shot	Bitti $\rightarrow$ idle
hurt	One-Shot	Bitti $\rightarrow$ idle (can $>$ 0)
death	One-Shot	Bitti $\rightarrow$ kill()

Öncelik Sırası

death $>$ hurt $>$ attack $>$ walk $>$ idle

Zamanlayıcılar

get_ticks() ve Delta Time

```
# Gecen sure (ms)
simdi = pygame.time.get_ticks()

# Delta time hesaplama
dt_ms = saat.tick(60)     # FPS siniri + gecen sure
dt = dt_ms / 1000.0       # Saniyeye cevir

# FPS bagimsiz hareket
hiz = 300  # piksel/saniye
self.x += hiz * dt
```

Neden Delta Time?

60 FPS'te: $300 \times 0.0167 \approx 5$ px/frame

30 FPS'te: $300 \times 0.0333 \approx 10$ px/frame

Her iki durumda da saniyede 300 piksel

USEREVENT ve set_timer()

```
# Ozel olaylar tanimla
DUSMAN_SPAWN = pygame.USEREVENT + 1
BONUS_SPAWN = pygame.USEREVENT + 2

# Periyodik zamanlayici
pygame.time.set_timer(DUSMAN_SPAWN, 800)    # 800 ms
pygame.time.set_timer(BONUS_SPAWN, 5000)    # 5 saniye

# Durdurmak icin
pygame.time.set_timer(DUSMAN_SPAWN, 0)
```

Sınır

En fazla 32 özel olay (USEREVENT + 0 $\sim$ 31)

Cooldown Sistemi

Cooldown Sınıfı

```
class Cooldown:
    def __init__(self, sure_ms):
        self.sure = sure_ms
        self.son_kullanim = 0

    def dene(self):
        simdi = pygame.time.get_ticks()
        if simdi - self.son_kullanim >= self.sure:
            self.son_kullanim = simdi
            return True
        return False
```

Cooldown Kullanımı

```
cd_mermi = Cooldown(200)      # 200 ms
cd_roket = Cooldown(2000)     # 2 saniye

if cd_mermi.dene():
    mermi_at()
```

Tipik Cooldown Süreleri

Mermi: 150–300 ms Roket: 1500–3000 ms Kalkan: 5000+ ms

Görsel Efektler

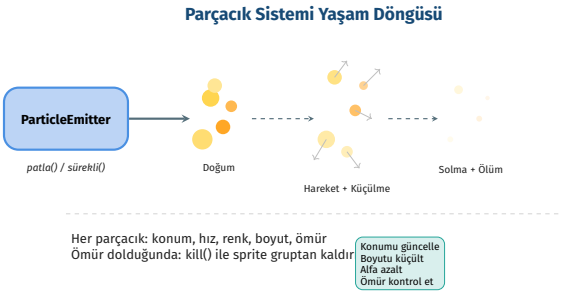
Parçacık Sistemi

Particle

- Konum, hız, renk, boyut, ömür
- Zamanla küçülür ve soluklaşır
- Ömür dolduğunda kill()

ParticleEmitter

- Merkez noktasından üretir
- Rastgele yön ve hız
- patla() → tek seferlik



Patlama = Sprite + Parçacık

```
def patlama_olustur(x, y, grup):
    # 1. Sprite patlama animasyonu
    patlama = Explosion(x, y, boyut=64)
    grup.add(patlama)

    # 2. Parcacik efekti
    renkler = [(255, 220, 100), (255, 150, 50)]
    emitter = ParticleEmitter(x, y, renkler, grup,
                              parcacik_sayisi=15)
    emitter.patla()
```

Juice

Patlama + Parçacık + Screen Shake + Ses = Profesyonel his

Screen Shake

```
class ScreenShake:
    def __init__(self, siddet=10, sure_ms=300):
        self.siddet = siddet
        self.sure = sure_ms

    def guncelle(self):
        ilerleme = gecen / self.sure
        mevcut_siddet = self.siddet * (1.0 - ilerleme)
        s = int(mevcut_siddet)
        self.ofset_x = random.randint(-s, s)
        self.ofset_y = random.randint(-s, s)
```

Kullanım

```
ekran.blit(sahne, (shake.ofset_x, shake.ofset_y))
```

Uzay Kaçışı v4

v4'te Eklenen Özellikler

1. AnimatedSprite altyapısı
2. Motor ateşi animasyonu (oyuncu)
3. Dönen bonus nesneleri
4. Cooldown sınıfı ile mermi bekleme
5. Dalga sistemi (artan zorluk)
6. Sprite + parçacık patlama efekti
7. Screen shake (hasar alınca)
8. Blink efekti (dokunulmazlık)

Versiyon Geçmişi

v1 (Sprite) → v2 (Çarpışma) → v3 (Ses) → **v4 (Animasyon + Efekt)**

Özet

Hafta 8 Özeti

Öğrenilenler

- Sprite sheet'ten frame kesme ve zamana bağlı animasyon
- AnimatedSprite sınıfı ve state machine tasarımı
- get_ticks(), USEREVENT, set_timer() ile zamanlama
- Cooldown sınıfı ile eylem bekleme süreleri
- Dalga sistemi ve zorluk artışı
- Parçacık, patlama, screen shake, blink efektleri

Gelecek Hafta

Bölüm 9: Metin, UI ve Menü Sistemleri